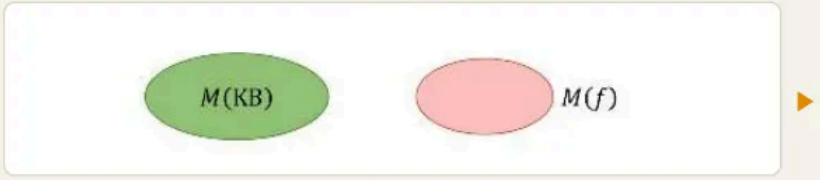
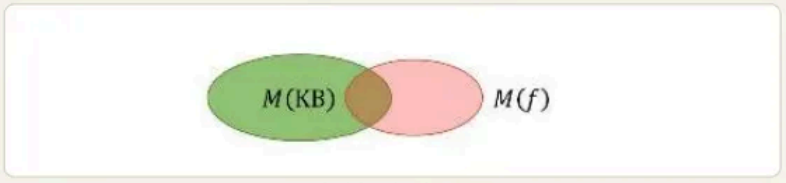


① Entailment: M(KB)⊆M(f)



② Contradiction: 교집합=∅



③ Contingency: 부분 교집합

세 가지 관계 — Venn 다이어그램 비교 (p.29-31)

Tell & Ask Operations p.33-34

Q. Tell·Ask 연산의 의미는?

Q. 새 정보의 3가지 분류는?

Tell[f] — 정보 입력 — KB에 formula f를 알림. 세 가지 응답:

- Entailment: KB ⊨ f → 이미 알고 있음
- Contradiction: KB ⊨ ¬f → 믿지 않음
- Contingent: KB를 f로 업데이트 (새 정보 학습)

Ask[f] — 질의 — KB에 f를 질의. 세 가지 응답:

- Yes: entailment (KB ⊨ f)
- No: contradiction (KB ⊨ ¬f)
- 모름: contingent

> 💡 **TIP** Tell ↔ Ask의 대칭성: 같은 KB를 공유하며, 둘 다 entailment 판단으로 귀결됨. 핵심은 KB ⊨ f를 효율적으로 계산하는 것.

Satisfiability & Model Checking p.36-39

Q. SAT 문제 정의는?

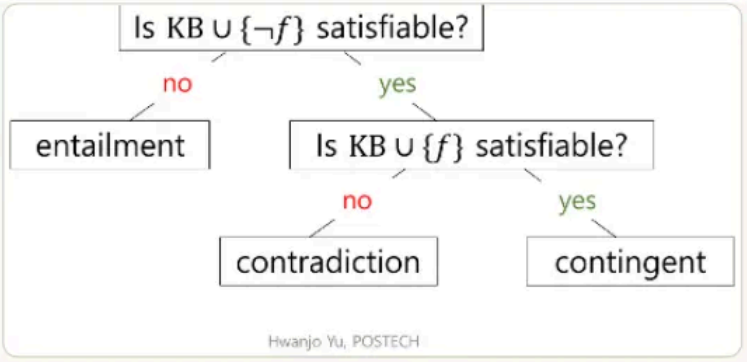
Q. Ask/Tell의 2-step SAT 환원?

Q. model checking 동작은?

Satisfiability 정의 — KB가 satisfiable: M(KB) ≠ ∅ (참인 세계가 하나 이상 존재).

Ask/Tell → SAT로 환원 — entailment/contradiction/contingency 판단을 모두 SAT 문제로 바꿀 수 있음.

- "Is KB∪{¬f} satisfiable?" 로 시작 → no: entailment / yes: 두 번째 질문으로
- "Is KB∪{f} satisfiable?" → no: contradiction / yes: contingent



Ask[f] → SAT 환원 결정 트리 (p.36)

Model Checking — SAT는 CSP의 특수 사례 — propositional symbol → CSP variable, formula → constraint, model → assignment.

- 대표 알고리즘: **DPLL** (backtracking + pruning), **WalkSAT** (randomized local search)
- SAT는 NP-complete — 최악 지수 시간이나 실용적 solver는 빠름

> ★ **필수암기** Syntax = 유효 formula 정의(기호, 의미 없음). Semantics = model w에서 I(f,w)∈{0,1} 계산(진리표+재귀). M(f)=formula를 만족하는 model 집합. KB ⊨ f ⟺ M(KB)⊆M(f). Contradiction ⟺ KB ⊨ ¬f. SAT: KB satisfiable ⟺ M(KB)≠∅. Ask/Tell 모두 SAT로 환원 가능.

요약 Propositional Logic: **Syntax(재귀 formula) → Semantics(I(f,w) 재귀평가) → KB(model 집합 교집합)** · Entailment/Contradiction/Contingency(Venn) · Ask/Tell → **SAT/model checking**으로 결정.

Ch 18 Logic 2 — First-Order Logic

명제 논리 추론 규칙 (Soundness · Completeness · Horn · Resolution) + FOL 소개

추론 프레임워크 p.4-6

Q. inference rule(⊢)의 역할은?

Q. derivation(증명)이란?

Inference Rule 정의 — 전제(premises) $\varphi_1,\cdots,\varphi_n$ 으로부터 결론 ψ 를 도출하는 구문적 규칙. 의미(semantics) 아닌 구문(syntax)에만 작동.

- **Modus ponens**: $\varphi,\ \varphi\to\psi \vdash \psi$ — 가장 기본적인 추론 규칙
- 일반형: $\varphi_1,\cdots,\varphi_n$ (premises) / ψ (conclusion)

Forward Inference (전방 추론)

```
Input: Rules, KB
repeat until no changes to KB:
  choose φ1,…,φn ∈ KB
  if matching rule φ1,…,φn / ψ exists:
    add ψ to KB
```

KB ⊢ ψ (derivation): ψ가 KB에 결국 추가되면 '유도됨(derives)'

Modus Ponens 예시

- KB = {Rain, Rain→Wet, Wet→Slippery}
- Step 1: Rain + Rain→Wet → Wet 추가
- Step 2: Wet + Wet→Slippery → Slippery 추가
- 한계: ¬Wet, Wet∧Slippery 는 도출 불가 — 규칙의 표현력 한계

Soundness & Completeness p.8-15 · 핵심 두 성질

Q. soundness {⊢}⊆{⊨} 의미는?

Q. completeness {⊢} ⊇{⊨} 의미는?

Q. 둘의 차이를 한 줄로?

Entailment vs Derivation — 의미론적 귀결: KB ⊨ φ (모든 model에서 true). 구문론적 유도: KB ⊢ φ (추론 규칙으로 도달). 두 집합의 포함 관계가 **soundness/completeness**를 정의.

sound → 거짓 없음
complete → 빠짐없음

- **Soundness**: {φ: KB ⊢ φ} ⊆ {φ: KB ⊨ φ} → "nothing but the truth" — 유도한 것은 모두 참
- **Completeness**: {φ: KB ⊢ φ} ⊇ {φ: KB ⊨ φ} → "the whole truth" — 참인 것은 모두 유도

★ 필수암기 Soundness = {⊢} ⊆ {⊨} (유도 ⊂ 귀결) · Completeness = {⊢} ⊇ {⊨} (유도 ⊃ 귀결). Modus ponens alone: sound ✓, complete ✗

Soundness 검증 — Modus Ponens

- 질문: Rain ∩ Rain→Wet 가 Wet 을 entail하는가?
- → Entailed, thus sound!
- 반례: Wet ∩ Rain→Wet 로 Rain 을 유도하는 규칙 → Not entailed → unsound

Completeness 반례 — Modus Ponens 불완전

- KB = {Rain, Rain∨Snow→Wet}, φ = Wet
- 의미적: KB ⊨ Wet (Rain이 true이므로 Rain∨Snow true → Wet true)
- 구문적: KB ⊢/Wet (Modus ponens로 도달 불가) → Incomplete!

	Modus Ponens	Resolution
Formula 제한	Horn clauses만	임의 clause
Completeness	완전 (Horn 한정)	완전 (일반)
시간복잡도	선형 O(n)	지수 O(exp)
표현력	덜 표현적	더 표현적

추론 규칙 트레이드오프 (p.23, 34)

Horn Clauses & Modus Ponens 완전성 p.18-22

Q. Horn clause의 형태는?

Q. modus ponens 규칙은?

Q. Horn에서만 완전한 이유·복잡도?

Definite Clause 정의 — $(p_1 \wedge \cdots \wedge p_n) \rightarrow q$ 형태. $p_1,\cdots,p_n,q$ 는 **명제 기호**.

- 예: (Rain ∧ Snow) → Traffic ✓ · Traffic ✓ (body 없음)
- 비예: Rain ∧ Snow ✗ · ¬Traffic ✗ · (Rain ∧ Snow) → (Traffic ∨ Peaceful) ✗
- ⟹ *head는 단일 양의 기호*, body는 conjunct만 허용

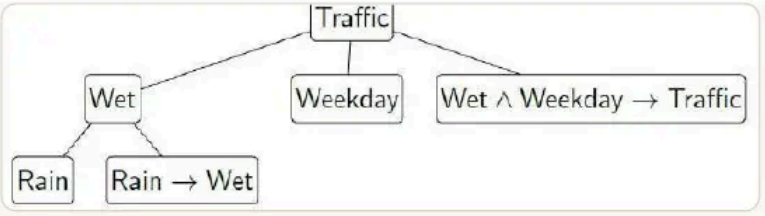
Horn Clause 정의 — Definite clause 또는 goal clause ($p_1 \wedge \cdots \wedge p_n \rightarrow$ false) 의 합집합.

- Definite: (Rain ∧ Snow) → Traffic
- Goal: Traffic ∧ Accident → false [≡ ¬(Traffic ∧ Accident)]
- 이접(disjunction) 표현: 각 Horn clause = **음수 literal 여럿 + 양수 literal 최대 1개**의 절(clause)

Modus Ponens (일반형)

- $p_1,\cdots,p_n$, $(p_1 \wedge \cdots \wedge p_n) \rightarrow q \vdash q$
- 예: Wet, Weekday, Wet∧Weekday→Traffic ⊢ Traffic
- 이접 재작성: p, ¬p∨q ⊢ q — p와 ¬p가 **상쇄(cancel)**

✎ 핵심 정리: Modus ponens on Horn clauses = **완전(complete)**. KB가 Horn clause만 포함하고 φ가 entailed 명제 기호이면, modus ponens가 반드시 φ를 도출. 즉 KB ⊨ φ ⟺ KB ⊢ φ (Horn KB에서만).



Modus ponens derivation tree: 잎=원래 KB 공식, 내부=modus ponens 결과 (p.22)

Resolution 추론 규칙

p.25-32 · 일반 clause · 지수 시간

Q. resolution 규칙은?

Q. CNF 변환 5단계는?

Q. refutation(¬φ 추가)의 원리는?

Literal & Clause

- **Literal**: 명제 기호 p 또는 그 부정 ¬p
- **Clause**: literal의 이접(disjunction) — ¬A∨B∨¬C∨D∨¬E∨F
- **Horn clause**: 양의 literal 최대 1개인 clause

Resolution Rule 정의 — 두 clause 에서 상호 보완 literal (ℓ 과 $\neg\ell$)을 제거해 새 clause를 생성.

- $(\ell_1 \vee \cdots \vee \ell_k \vee \ell),\ (\neg\ell \vee m_1 \vee \cdots \vee m_n) \vdash \ell_1 \vee \cdots \vee \ell_k \vee m_1 \vee \cdots \vee m_n$
- 예: Rain∨Snow, ¬Snow∨Traffic ⊢ Rain∨Traffic
- **Modus ponens**: resolution의 특수 케이스 — p, ¬p∨q ⊢ q
- Soundness: Rain∨Snow ∩ ¬Snow∨Traffic ⊆ (Rain∨Traffic) → Entailed, sound

> ★ 필수암기 Resolution = 완전(complete) + 건전(sound) for 명제논리. 단, 시간복잡도 {지수} — clause가 많은 literal 포함 가능 → 지수적 확장.

CNF (Conjunctive Normal Form) — clause들의 conjunction(∧). Resolution은 CNF에서만 직접 작동.

- 예: (A∨B∨¬C) ∧ (¬A∨B)
- 정리: 모든 명제논리 공식 φ는 동등한 CNF φ' 로 변환 가능 — 표현력 손실 없음

CNF 변환 알고리즘 (5단계)

```
1. Eliminate ↔ (biconditional):
   φ↔ψ  →  (φ→ψ)∧(ψ→φ)
2. Eliminate → (implication):
   φ→ψ  →  ¬φ∨ψ
3. Move ¬ inwards (de Morgan):
   ¬(φ∧ψ)  →  ¬φ∨¬ψ
   ¬(φ∨ψ)  →  ¬φ∧¬ψ
4. Eliminate double negation:
   ¬¬φ  →  φ
5. Distribute ∨ over ∧:
   φ∨(ψ∧χ)  →  (φ∨ψ)∧(φ∨χ)
```

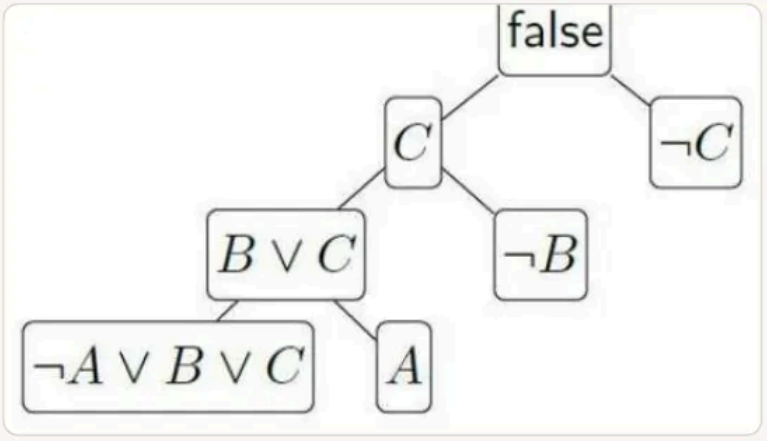
예: (Summer→Snow)→Bizzare → (Summer∨Bizzare)∧(¬Snow∨Bizzare)

Refutation (반박) 기반 Resolution — 핵심 관계: KB ⊨ φ ⟺ KB ∪ {¬φ} 가 unsatisfiable

Resolution-Based Inference

```
Input: KB, φ
1. Add ¬φ to KB
2. Convert all formulas to CNF
3. repeat:
     apply resolution rule to pairs of clauses
     add new clause to KB
4. return ENTAILED if derive false (empty clause)
   return UNKNOWN otherwise
```

false(빈 clause) 도출 = KB∪{¬φ}이 비일관(unsatisfiable) → KB⊨φ 증명됨



Resolution refutation tree 예시: false 도출 → KB entails C (p.32)

Resolution 예제 추적 — KB = {A→(B∨C), A, ¬B}, KB ⊨ C?

- Step 1: Add ¬C → KB' = {A→(B∨C), A, ¬B, ¬C}
- Step 2: CNF → KB' = {¬A∨B∨C, A, ¬B, ¬C}
- Step 3: resolve ¬A∨B∨C + A → B∨C
- Step 4: resolve B∨C + ¬B → C
- Step 5: resolve C + ¬C → false (빈 clause) → KB ⊨ C 증명

복잡도 & 최종 비교 p.33-34

Q. modus ponens vs resolution 비교?

Q. 완전성·복잡도 트레이드오프?

시간복잡도

- Modus ponens: 매 적용마다 단일 명제 기호 1개 추가 → 최대 n번 → O(n) 선형
- Resolution: 매 적용마다 많은 literal 포함 clause 추가 → O(exp) 지수

	Horn+Modus Ponens	Any+Resolution
Formula	Horn clauses만	임의 clause
완전성	완전	완전
시간	선형 O(n)	지수 O(exp)
표현력	덜 표현적	더 표현적

Horn+MP vs Resolution 트레이드오프

⚠ 함정 Modus ponens가 incomplete한 이유: Horn clause 형식을 벗어난 공식(예: A∨B→C)은 전제로 쓸 수 없음 → 참인 결론을 도달 못할 수 있음. 해결책 ①: Horn clauses로 제한(표현력↓) / ②: Resolution 사용(시간↑).

★ 필수암기 Modus ponens: sound+Horn완전·선형 · Resolution: sound+일반완전·지수 · CNF변환 5단계(↔ → ¬안으로 ¬¬제거 ∨/∧분배) · refutation: KB⊨φ ⟺ KB∪{¬φ} unsatisfiable

요약 **Soundness** $\{\vdash\} \subseteq \{\models\}$ · **Completeness** $\{\vdash\} \supseteq \{\models\}$ · Modus ponens는 **Horn clause**에서만 완전(선형) · Resolution은 임의 clause 완전하나 지수 · CNF 5단계 + refutation($KB \cup \{\neg\varphi\}$ unsat $\iff KB \models \varphi$).